

Más allá de la Base de Datos (Storage de Archivos y Serverless Functions)

1. Introducción y Objetivos

Hasta este punto hemos aprendido a autenticar usuarios y a manipular estructuras de datos primitivas como texto, números y booleanos en PostgreSQL. Sin embargo, las aplicaciones del mundo real requieren gestionar archivos binarios pesados (*BLOBs*) como fotografías de perfil, comprobantes PDF o archivos multimedia. Guardar estos archivos codificados directamente en texto (por ejemplo, en Base64) dentro de una base de datos relacional degrada drásticamente el rendimiento del motor SQL.

El objetivo de esta sesión es que entiendan el paradigma de **Almacenamiento de Objetos (Storage)** basado en **Buckets**. Además aprenderán a capturar recursos físicos desde el hardware del dispositivo móvil (Cámara), cargarlos eficientemente a la nube de Supabase y enlazar las URLs públicas resultantes con sus registros relacionales. Adicionalmente, se revisará a nivel conceptual el paradigma de **Edge Functions (FaaS)** como la solución arquitectónica estándar para ejecutar lógica pesada o sensible fuera del celular.

2. Fundamentos Técnicos

A. Anatomía del Almacenamiento de Objetos (Buckets)

A diferencia de un sistema de archivos tradicional en un sistema operativo, un **Object Storage** organiza los datos en contenedores planos llamados **Buckets**.

- **Políticas de Privacidad:** Un Bucket puede configurarse como **Público** (cualquier persona con el enlace puede descargar el archivo sin firmar peticiones) o **Privado** (requiere de tokens de acceso temporales firmados criptográficamente para su lectura).
- **Optimización Móvil:** El flujo correcto desde una aplicación nativa consiste en:
 1. Capturar el archivo localmente.
 2. Subirlo al Bucket de Storage para obtener un identificador único o URL.
 3. Almacenar únicamente esa cadena de texto corta (URL) en la tabla relacional de la base de datos.

B. Cómputo en la Periferia: Edge Functions (Deno / TypeScript)

Cuando una aplicación móvil necesita procesar un pago (ej. Stripe), enviar un correo masivo o validar reglas de negocio complejas que no deben ser visibles por ingeniería inversa en el código del cliente móvil, se recurre a la **Lógica Serverless**.

Supabase implementa **Edge Functions**, que son pequeños bloques de código escritos en **TypeScript** que se ejecutan en servidores distribuidos geográficamente en todo el mundo a través del motor **Deno Runtime**. No se administra un servidor web (como Apache o Nginx); el código "despierta" bajo demanda mediante una petición HTTP **POST/GET**, procesa la información y se apaga de inmediato, cobrando únicamente por los milisegundos de ejecución exacta.

3. Preparación en la Consola Web de Supabase (10 min)

Paso 1: Crear un Bucket Público para Imágenes

1. Abre tu consola del proyecto en supabase.com.
2. En el menú lateral izquierdo, haz clic en **Storage**.
3. Haz clic en **New Bucket**.
4. Nombra al bucket exactamente: **avatares**
5. Activa la opción de **Public bucket** (esto permite que las imágenes puedan ser mostradas en la app mediante una URL estándar sin lidiar hoy con tokens de descarga).
6. Haz clic en **Save**.

Paso 2: Modificar la base de datos para almacenar la URL

Ve al **SQL Editor** y ejecuta la siguiente consulta para añadir una columna a nuestra tabla previa:

```
-- Añadir columna para la URL de la imagen en la tabla existente de tareas
ALTER TABLE tareas ADD COLUMN imagen_url TEXT;
```

4. Laboratorio Código Fuente Paso a Paso

Para interactuar con el hardware de la cámara del dispositivo, añadiremos el plugin estándar de Flutter a las dependencias.

Paso 1: Añadir Dependencias de Control de Hardware

Ejecuta el siguiente comando en la terminal de tu proyecto:

```
flutter pub add image_picker
```

⚠ Nota Crítica para Android (MinSdkVersion): El paquete **image_picker** requiere que la app corra como mínimo en Android SDK 21. Como se configuró desde el "Hola Mundo", no debería haber problema, pero recuérdale al grupo verificar su archivo **android/app/build.gradle**.

Paso 2: Actualizar el Modelo de Datos

Modificamos nuestra entidad para dar soporte al nuevo campo de la base de datos PostgreSQL.

lib/domain/tarea_model.dart

```
class Tarea {
  final int? id;
  final String userId;
  final String titulo;
  final bool completado;
  final String? imageUrl; // <-- NUEVO
```

```

Tarea({
  this.id,
  required this.userId,
  required this.titulo,
  this.completado = false,
  this.imagenUrl, // <-- NUEVO
});

factory Tarea.fromMap(Map<String, dynamic> map) {
  return Tarea(
    id: map['id'] as int?,
    userId: map['user_id'] as String,
    titulo: map['titulo'] as String,
    completado: map['completado'] as bool,
    imagenUrl: map['imagen_url'] as String?, // <-- NUEVO
  );
}

Map<String, dynamic> toMap() {
  return {
    if (id != null) 'id': id,
    'user_id': userId,
    'titulo': titulo,
    'completado': completado,
    'imagen_url': imagenUrl, // <-- NUEVO
  };
}

```

Paso 3: Añadir la Lógica de Storage al Repositorio

Actualizaremos la interfaz y la implementación de nuestro repositorio de datos para procesar la subida de los archivos binarios del teléfono hacia el bucket en la nube.

`lib/infrastructure/supabase_tareas_repository.dart` (Modificación de Métodos)

```

import 'dart:io';
import 'package:supabase_flutter/supabase_flutter.dart';
// ... mantener el resto de imports previos

class SupabaseTareasRepository implements TareasRepository {
  final SupabaseClient _supabaseClient = Supabase.instance.client;

  // ... mantener métodos escucharTareas, actualizarEstadoTarea y eliminarTarea
  intactos

  /// Sube un archivo binario al Bucket y retorna su URL Pública de acceso
  Future<String> _subirImagenABucket(File archivoImagen) async {
    try {

```

```

        // Generamos un nombre único para el archivo basado en milisegundos
        final String nombreArchivo = '${DateTime.now().millisecondsSinceEpoch}.jpg';

        // Accedemos a la API de Storage de Supabase e insertamos los bytes del
        archivo
        await _supabaseClient.storage
            .from('avatares')
            .upload(nombreArchivo, archivoImagen);

        // Obtenemos la URL de descarga del archivo recién subido
        final String urlPublica = _supabaseClient.storage
            .from('avatares')
            .getPublicUrl(nombreArchivo);

        return urlPublica;
    } catch (e) {
        throw Exception('Fallo al cargar archivo al Storage: $e');
    }
}

@override
Future<void> crearTareaConImagen(Tarea tarea, File? archivoImagen) async {
    try {
        String? urlFinal;

        // Si el alumno seleccionó una foto, primero la procesamos en el bucket
        if (archivoImagen != null) {
            urlFinal = await _subirImagenABucket(archivoImagen);
        }

        // Creamos el clon del objeto con su URL correspondiente
        final nuevaTareaConUrl = Tarea(
            userId: tarea.userId,
            titulo: tarea.titulo,
            imagenUrl: urlFinal,
        );

        // Insertamos el registro en PostgreSQL
        await _supabaseClient.from('tareas').insert(nuevaTareaConUrl.toMap());
    } catch (e) {
        throw Exception('Error al guardar la tarea relacional: $e');
    }
}
}

```

Paso 4: Ajuste de Interfaz de Usuario (Captura y Renderizado)

Actualizaremos la vista `HomeScreen` para permitir disparar la cámara y mostrar miniaturas de imágenes en la nube utilizando cacheo o renderizado HTTP.

`lib/presentation/home_screen.dart` (Sección del Formulario)

```
import 'dart:io';
import 'package:flutter/material';
import 'package:image_picker/image_picker.dart';
// ... conservar imports de modelos y repositorios previos

class _HomeScreenState extends State<HomeScreen> {
  final SupabaseTareasRepository _tareassRepository = SupabaseTareasRepository();
  final _todoController = TextEditingController();
  File? _imagenSeleccionada; // Variable de estado para la foto temporal
  bool _isUploading = false;

  // Método para disparar la cámara del dispositivo móvil
  Future<void> _capturarFoto() async {
    final picker = ImagePicker();
    final XFile? foto = await picker.pickImage(
      source: ImageSource.camera, // Cambiar a ImageSource.gallery si prueban en
emulador sin cámara
      imageQuality: 70,           // Optimización: reduce el peso comprimiendo al
70%
    );

    if (foto != null) {
      setState(() {
        _imagenSeleccionada = File(foto.path);
      });
    }
  }

  void _guardarTareaCompleta() async {
    if (_todoController.text.trim().isEmpty) return;

    setState(() => _isUploading = true);

    final plantillaTarea = Tarea(
      userId: widget.userId,
      titulo: _todoController.text.trim(),
    );

    try {
      await _tareassRepository.crearTareaConImagen(plantillaTarea,
_imagenSeleccionada);
      _todoController.clear();
      setState(() => _imagenSeleccionada = null); // Limpiar preview
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.toString()), backgroundColor: Colors.redAccent),
      );
    } finally {
      setState(() => _isUploading = false);
    }
  }
}
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Evidencias en la Nube (Storage)'),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          // Panel de Control del Formulario
          Card(
            child: Padding(
              padding: const EdgeInsets.all(12.0),
              child: Column(
                children: [
                  TextField(
                    controller: _todoController,
                    decoration: const InputDecoration(labelText: 'Descripción de
la evidencia...'),
                  ),
                  const SizedBox(height: 12),
                  Row(
                    mainAxisAlignment: MainAxisAlignment.spaceBetween,
                    children: [
                      IconButton(
                        onPressed: _capturarFoto,
                        icon: const Icon(Icons.camera_alt),
                        label: Text(_imagenSeleccionada == null ? 'Tomar
Evidencia' : '¡Foto Lista!'),
                      ),
                      if (_isUploading)
                        const CircularProgressIndicator(color: Colors.emerald)
                      else
                        ElevatedButton(
                          onPressed: _guardarTareaCompleta,
                          child: const Text('Subir Todo'),
                        )
                    ],
                  ),
                  if (_imagenSeleccionada != null) ...[
                    const SizedBox(height: 8),
                    Image.file(_imagenSeleccionada!, height: 100, fit:
BoxFit.cover)
                  ]
                ],
              ),
            ),
          ),
          const SizedBox(height: 16),
          // El listado reactivo (StreamBuilder) se mantiene igual a la sesión
2,
          // solo modificamos la celda (ListTile) para renderizar la imagen web.
          Expanded(
            child: StreamBuilder<List<Tarea>>(
              stream: _tarearRepository.escucharTareas(widget.userId),

```

```

        builder: (context, snapshot) {
          if (!snapshot.hasData || snapshot.data!.isEmpty) return const
Center(child: Text('Sin registros.'));
          final tareas = snapshot.data!;
          return ListView.builder(
            itemCount: tareas.length,
            itemBuilder: (context, index) {
              final t = tareas[index];
              return ListTile(
                title: Text(t.titulo),
                // RENDER DE IMAGEN DESDE EL BUCKET SI EXISTE LA URL
                leading: t.imagenUrl != null
                  ? Image.network(t.imagenUrl!, width: 50, height: 50,
fit: BoxFit.cover)
                  : const Icon(Icons.image_not_supported, color:
Colors.grey),
                trailing: IconButton(
                  icon: const Icon(Icons.delete, color: Colors.red),
                  onPressed: () => _tarearsRepository.eliminarTarea(t.id!),
                ),
              ),
            ),
          ),
        ),
      ],
    ),
  ),
);
}
}

```

5. Anexos

Si se presentan mensajes de error de 'acceso no autorizado', prueben agregando esto desde el editor de SQL:

```

-- 1. Permitir que los usuarios autenticados puedan subir (INSERT) fotos al bucket
'avatares'
CREATE POLICY "Permitir subida de avatares a usuarios autenticados"
ON storage.objects
FOR INSERT
TO authenticated
WITH CHECK (bucket_id = 'avatares');

-- 2. (Opcional) Permitir que los usuarios puedan borrar sus propios archivos si
lo necesitan
CREATE POLICY "Permitir borrado de avatares a usuarios autenticados"
ON storage.objects

```

```
FOR DELETE  
TO authenticated  
USING (bucket_id = 'avatares');
```